

Analiza experimentală comparativă a algoritmilor de sortare

Studiu asupra influenței structurii de date, distribuției inputului și
implementării asupra performanței

Butoi Alexandru-Gabriel
Universitatea de Vest din Timișoara
Facultatea de Informatică
Specializarea Informatică, Română
[alexandru.butoi06@e-uvt.ro]

2026

Rezumat

Lucrarea prezintă o analiză experimentală comparativă a mai multor algoritmi de sortare, implementați și testați pe structuri de date diferite: **vector**, **array** și listă simplu înlanțuită. Studiul urmărește relația dintre complexitatea teoretică, comportamentul practic al algoritmilor, localitatea în memorie, distribuția datelor de intrare și calitatea concretă a implementării.

Problema cercetată este alegerea unei metode eficiente de sortare în condiții practice, în care performanța nu depinde exclusiv de ordinul asimptotic al algoritmului, ci și de structura de stocare, de costurile auxiliare, de modul de acces la memorie și de particularitățile inputului. Rezultatele experimentale confirmă diferența majoră dintre algoritmii de complexitate quadratică și cei de complexitate $O(n \log n)$, dar evidențiază și faptul că algoritmi teoretic similari pot avea performanțe foarte diferite în practică.

Lucrarea include formularea problemei, analiza audienței, definirea obiectivelor operaționale, prezentarea cadrului metodologic, interpretarea rezultatelor experimentale, comparația cu literatura de specialitate și direcțiile viitoare de cercetare.

Cuvinte-cheie: algoritmi de sortare, complexitate, QuickSort, MergeSort, IntroSort, structuri de date, localitate în memorie, analiză experimentală.

Cuprins

1	Introducere	4
1.1	Problema cercetată	4
1.2	Demersul de cercetare	4
1.3	Structura lucrării	5
2	Analiza audienței	5
2.1	Nivelul de cunoștințe	5
2.2	Nivelul de interes	6
2.3	Așteptările audienței	6
3	Obiective operaționale și aspecte cognitive ale învățării	6
3.1	Motivația	7
3.2	Învățarea	7
3.3	Asocierea	7
4	Literatura de specialitate și rolul surselor bibliografice	8
5	Metodologia cercetării	9
5.1	Algoritmii analizați	10
5.2	Tipuri de distribuții ale datelor	10
5.3	Separarea timpului de sortare de costul I/O	11
5.4	Mediul de execuție	11
6	Modelare și implementare	12
6.1	Modelul general al experimentului	12
6.2	Structuri de date	13
6.3	Algoritmi de complexitate quadratică	13
6.4	Algoritmi de tip $O(n \log n)$	13
6.5	Implementări manuale și implementări standard	13

7	Rezultate experimentale	14
7.1	Algoritmi de complexitate $O(n^2)$	14
7.2	Algoritmi de complexitate $O(n \log n)$	15
7.3	Comportamentul structurilor de date	15
8	Interpretarea rezultatelor	16
8.1	De ce Merge Sort pe listă pierde timp față de Merge Sort pe array	16
8.2	De ce Merge Sort paralel pe 16 thread-uri pierde față de QuickSort cu pivot random	17
8.3	De ce implementarea manuală de IntroSort este inferioară lui <code>std::sort</code> .	18
8.4	Sensibilitatea la distribuția datelor	20
8.5	Normalizarea tipurilor de date: conversia la întregi	20
9	Comparație cu literatura de specialitate	21
10	Direcții viitoare de cercetare	22
10.1	Analiza sensibilității algoritmilor la distribuția datelor	22
10.2	Studiul efectului localității în memorie	22
10.3	Praguri optime și algoritmi hibridi	23
10.4	Strategii alternative de alegere a pivotului în QuickSort	23
10.5	Raportul dintre paralelizare și beneficiul real de performanță	24
10.6	Păstrarea datelor pentru validare și cercetare ulterioară	24
11	Concluzii finale	25

1. Introducere

Sortarea reprezintă una dintre problemele fundamentale ale informaticii, având aplicații directe în procesarea datelor, baze de date, sisteme de fișiere, algoritmi de căutare, analiză numerică și aplicații software de scară mare. Deși algoritmi de sortare sunt studiați frecvent din perspectivă teoretică, performanța lor practică depinde de mai mulți factori care nu sunt surprinși integral de notația asimptotică.

În literatura de specialitate, algoritmi de sortare sunt clasificați, de regulă, în funcție de complexitatea temporală, complexitatea spațială, stabilitate, adaptabilitate la input și mod de funcționare [2, 5, 7]. Totuși, într-un program real, două implementări ale aceluiași algoritm pot produce timpi diferiți, iar doi algoritmi cu aceeași complexitate teoretică pot avea performanțe practice semnificativ diferite.

Din acest motiv, lucrarea de față nu urmărește doar prezentarea teoretică a algoritmilor, ci mai ales analiza lor experimentală. Scopul este de a observa cum se comportă algoritmi atunci când sunt aplicați pe volume diferite de date, pe distribuții diferite ale inputului și pe structuri de date diferite.

1.1. Problema cercetată

Problema centrală a acestei lucrări poate fi formulată astfel:

În ce măsură alegerea algoritmului de sortare, a structurii de date și a implementării concrete influențează timpul real de execuție?

În mod teoretic, algoritmi pot fi clasificați în funcție de complexitatea lor, de exemplu $O(n^2)$ sau $O(n \log n)$. Totuși, în practică, această clasificare nu este suficientă pentru a anticipa complet comportamentul unui program. Diferențele pot apărea din cauza localității în memorie, a numărului de copieri, a costurilor de recursie, a accesului secvențial sau direct și a modului în care datele sunt distribuite.

Prin urmare, lucrarea urmărește nu doar confirmarea teoriei, ci și evaluarea comportamentului real al algoritmilor în condiții experimentale controlate.

1.2. Demersul de cercetare

Realizarea lucrării urmează un demers logic specific unei cercetări aplicate:

- 1) **Înțelegerea problemei:** identificarea sortării ca problemă fundamentală și stabilirea factorilor care pot influența performanța practică.

- 2) **Căutarea în literatura de specialitate:** consultarea lucrărilor clasice și a surselor tehnice relevante pentru algoritmi analizați.
- 3) **Evaluarea soluțiilor existente:** analizarea algoritmilor consacrați și a implementărilor standard, în special a funcției `std::sort`.
- 4) **Elaborarea unei soluții proprii:** construirea unui cadru experimental de testare, rularea algoritmilor pe date variate și interpretarea rezultatelor obținute.

1.3. Structura lucrării

Lucrarea este organizată conform formatului explicit al unei lucrări științifice. Introducerea formulează problema, justifică tema și stabilește direcția cercetării. Conținutul principal cuprinde partea formală, partea de modelare și implementare, respectiv partea experimentală. Închiderea lucrării include comparația cu literatura de specialitate, concluziile și direcțiile viitoare de cercetare.

Având în vedere că o lucrare de acest tip depășește, în mod normal, timpul de lectură al unui material scurt, conținutul este împărțit în secțiuni clare, pentru a permite cititorului să urmărească progresiv trecerea de la problemă la experiment și de la experiment la concluzii.

2. Analiza audienței

Lucrarea se adresează unui public cu nivel mediu și avansat de cunoștințe în informatică, în special studenților care au parcurs noțiuni fundamentale de programare, structuri de date și complexitate algoritmică.

2.1. Nivelul de cunoștințe

Publicul țintă trebuie să cunoască noțiuni precum:

- complexitate asimptotică;
- structuri de date liniare;
- recursivitate;
- pointeri și alocare dinamică;
- diferența dintre analiză teoretică și măsurare experimentală;
- noțiuni de bază privind compilarea și rularea unui program C++.

Lucrarea poate fi parcursă și de un cititor aflat la început de drum, însă interpretarea completă a rezultatelor presupune familiaritate cu algoritmi clasici de sortare.

2.2. Nivelul de interes

Interesul principal al audienței este înțelegerea diferenței dintre performanța teoretică și performanța practică. Într-un curs de algoritmică, este ușor să se afirme că un algoritm este $O(n^2)$, iar altul este $O(n \log n)$. Într-un program real, însă, diferențele se văd în timpi de execuție, consum de memorie, comportament pe cache și stabilitate în raport cu distribuția datelor.

Lucrarea urmărește să aducă cititorul pe subiect printr-o întrebare practică: de ce un algoritm care pare corect teoretic poate deveni inutilizabil pentru volume mari de date?

2.3. Așteptările audienței

Audiența se așteaptă ca lucrarea să ofere:

- explicații clare privind diferențele dintre algoritmi;
- rezultate experimentale relevante;
- comparații între structuri de date;
- concluzii aplicabile în programare practică;
- o legătură explicită între teorie, implementare și performanță;
- direcții de optimizare și cercetare ulterioară.

Prin urmare, lucrarea trebuie să fie suficient de formală pentru a susține concluziile, dar și suficient de clară pentru a permite înțelegerea fenomenelor observate.

3. Obiective operaționale și aspecte cognitive ale învățării

Obiectivele operaționale ale lucrării sunt formulate astfel încât cititorul să poată înțelege, analiza și aplica rezultatele obținute.

La finalul parcurgerii lucrării, cititorul trebuie să fie capabil să:

- 1) explice diferența dintre complexitatea teoretică și performanța practică a unui algoritm;

- 2) compare algoritmi de sortare din clase diferite de complexitate;
- 3) identifice impactul structurii de date asupra timpului de execuție;
- 4) interpreteze rezultatele experimentale în raport cu literatura de specialitate;
- 5) justifice alegerea unui algoritm de sortare într-un context concret;
- 6) explice de ce localitatea în memorie poate influența performanța;
- 7) propună direcții de optimizare sau extindere a experimentului.

3.1. Motivația

Motivația lucrării pornește de la o observație practică: în programare, alegerea algoritmului potrivit poate avea un impact major asupra timpului de execuție. Diferența dintre un algoritm quadratic și unul de tip $O(n \log n)$ poate transforma o problemă rezolvabilă în câteva secunde într-una imposibil de rulat într-un timp acceptabil.

Prin urmare, lucrarea oferă cititorului un motiv clar pentru parcurgerea materialului: înțelegerea performanței reale a algoritmilor este esențială pentru scrierea unui cod eficient. Nu este suficient ca un algoritm să fie corect; el trebuie să fie și practic utilizabil pentru dimensiunea reală a datelor.

3.2. Învățarea

Procesul de învățare urmărit prin această lucrare nu constă doar în memorarea complexităților algoritmilor, ci în corelarea acestora cu rezultate experimentale concrete. Cititorul este ghidat să observe cum aceeași problemă, sortarea, poate avea comportamente radical diferite în funcție de algoritm, structură de date și distribuția inputului.

Astfel, învățarea se produce prin trecerea de la nivelul abstract la nivelul măsurabil: de la formule de complexitate la timpi reali de execuție.

3.3. Asocierea

Prin asocierea dintre teorie și experiment, cititorul poate înțelege utilitatea practică a noțiunilor studiate. Rezultatele obținute arată că ceea ce se învață la nivel teoretic are consecințe directe în implementarea reală.

Această asociere produce un câștig cognitiv important: cititorul nu rămâne doar cu ideea că un algoritm este mai bun decât altul, ci înțelege de ce este mai bun, când este mai bun și ce limite poate avea în practică. În acest sens, cunoașterea algoritmilor permite rezolvarea unor probleme mai mari, mai rapid și cu resurse mai bine controlate.

4. Literatura de specialitate și rolul surselor bibliografice

Bibliografia utilizată în această lucrare are două roluri principale: fundamentarea teoretică a algoritmilor analizați și atribuirea corectă a contribuțiilor originale.

Nu este corect ca toți algoritmi de sortare să fie citați exclusiv printr-o singură sursă generală, cum ar fi Cormen et al. [2]. O astfel de sursă este utilă pentru prezentarea modernă și didactică a algoritmilor, dar nu înlocuiește lucrările originale sau sursele specializate. QuickSort trebuie asociat cu lucrarea originală a lui Hoare [4], Heapsort cu Williams [8], iar IntroSort cu Musser [6].

Sursele bibliografice pot fi împărțite în trei categorii:

- 1) **Surse teoretice generale**, folosite pentru complexitate, clasificări și explicații algoritmice: Cormen et al. [2], Knuth [5], Sedgewick și Wayne [7].
- 2) **Surse originale sau specializate**, folosite pentru creditarea algoritmilor și a ideilor specifice: Hoare [4], Williams [8], Musser [6], Bentley și McIlroy [1].
- 3) **Surse tehnice și experimentale**, folosite pentru raportarea comportamentului implementărilor moderne și pentru susținerea rezultatelor proprii: documentația pentru `std::sort` [3] și rapoartele experimentale generate în cadrul lucrării [?].

Algoritm / concept	Sursă principală	Rolul sursei în lucrare
Bubble Sort, Selection Sort, Insertion Sort	Knuth [5], Sedgewick și Wayne [7], Cormen et al. [2]	Surse generale pentru algoritmi clasici, complexitate și comportament didactic. Pentru acești algoritmi elementari nu este atribuit un unic inventator cert.
Merge Sort	Knuth [5], Cormen et al. [2]	Fundamentare teoretică pentru metoda divide et impera, interclasare și analiză de complexitate.
QuickSort	Hoare [4]	Lucrarea originală asociată algoritmului QuickSort. Sursa este folosită pentru creditarea corectă a algoritmului.

Algoritm / con- cept	Sursă principală	Rolul sursei în lucrare
HeapSort	Williams [8]	Lucrarea originală pentru HeapSort, relevantă inclusiv pentru înțelegerea componentei HeapSort din IntroSort.
IntroSort	Musser [6]	Lucrarea originală care introduce algoritmul introspectiv, folosit ca fundament pentru comparația cu implementările moderne de tip <code>std::sort</code> .
Implementări eficiente de sortare	Bentley și McIlroy [1]	Sursă relevantă pentru ideea că performanța unei funcții de sortare depinde decisiv de detaliile de implementare.
<code>std::sort</code>	cppreference [3]	Sursă tehnică folosită pentru raportarea cerințelor moderne ale funcției standard de sortare din C++.
Rezultatele experimentale proprii	Rapoartele generate de autor [?]	Sursa primară pentru datele măsurate, graficele construite și concluziile experimentale ale lucrării.

Prin această împărțire, bibliografia nu este folosită doar formal. Fiecare sursă are un rol explicit: unele explică teoria, altele oferă credit istoric, iar rezultatele proprii susțin partea experimentală.

5. Metodologia cercetării

Metodologia cercetării este experimentală și comparativă. Pentru fiecare algoritm au fost colectați timpi de execuție pentru același set de teste, iar pentru majoritatea implementărilor s-a urmărit separat comportamentul pe:

- vector;
- array;
- listă simplu înlănțuită.

Din perspectiva cercetării experimentale, această abordare este relevantă deoarece izolează două surse distincte de variație:

- a) variația datorată algoritmului propriu-zis;
- b) variația datorată structurii de date și localității în memorie.

Analiza nu se limitează la afirmații teoretice de forma $O(n^2)$ sau $O(n \log n)$, ci urmărește diferențele efective de timp, de ordinul nanosecundelor, milisecundelor, secundelor și, pentru cazurile nefavorabile, chiar al orelor sau zilelor estimate.

5.1. Algoritmii analizați

Au fost analizate următoarele familii de algoritmi:

- Bubble Sort;
- Selection Sort;
- Binary Insertion Sort;
- Merge Sort clasic;
- Merge Sort paralel;
- QuickSort;
- IntroSort.

Algoritmii Bubble Sort, Selection Sort și Binary Insertion Sort au fost tratați ca reprezentanți ai clasei $O(n^2)$. Merge Sort, QuickSort și IntroSort au fost tratați ca reprezentanți ai clasei $O(n \log n)$, cu observația că implementările concrete pot modifica semnificativ timpii obținuți.

5.2. Tipuri de distribuții ale datelor

Seturile de test au inclus mai multe tipuri de distribuții:

- date random;
- date deja sortate;
- date sortate invers;

- date egale;
- date aproape sortate;
- date cu multe duplicate;
- date binare sau cu interval numeric restrâns.

Această diversitate permite observarea sensibilității algoritmilor la forma inputului. În practică, dimensiunea n nu este singurul factor relevant; distribuția datelor poate influența decisiv numărul de comparații, numărul de mutări și comportamentul partiționării.

5.3. Separarea timpului de sortare de costul I/O

Măsurarea timpului a fost realizată prin izolarea operației de sortare. Citirea datelor este efectuată înainte de pornirea cronometrării, timerul este pornit exact înainte de apelul funcției de sortare, iar oprirea sa are loc imediat după finalizarea sortării, înainte de scrierea rezultatului în fișier.

Prin urmare, valorile măsurate reflectă timpul algoritmului de sortare propriu-zis, nu timpul combinat de I/O și procesare. Această alegere experimentală este importantă deoarece izolează variabila relevantă.

În cercetări viitoare, raportarea ar putea fi extinsă prin separarea explicită a:

- timpului de citire;
- timpului efectiv de sortare;
- timpului de scriere.

5.4. Mediul de execuție

Toate experimentele au fost realizate într-un mediu hardware și software controlat, pentru a asigura consistența și comparabilitatea rezultatelor obținute.

Testele au fost rulate pe următoarea configurație:

- Sistem de operare: Windows 11 Pro;
- Mediu de execuție: Windows Subsystem for Linux, utilizat prin Visual Studio Code;
- Procesor: Intel Core i7-13650HX;
- Memorie RAM: 16 GB, frecvență efectivă 4800 MT/s;

- Placă video: NVIDIA GeForce RTX 4060 Laptop GPU.

Placa video este menționată ca parte a configurației hardware generale, însă algoritmi testați au rulat pe procesor, nu pe GPU.

Alegerea utilizării WSL ca mediu de execuție a permis rularea codului într-un context apropiat de sistemele Unix-like, menținând în același timp stabilitatea și accesibilitatea mediului Windows. Toate testele au fost efectuate în aceleași condiții de execuție, fără variații ale configurației hardware sau software între rulări.

Din perspectivă experimentală, această consistență este esențială, deoarece permite compararea directă a algoritmilor fără influențe externe semnificative. Deși valorile absolute ale timpilor de execuție pot depinde de arhitectura sistemului, comparațiile relative dintre algoritmi rămân valide și relevante în cadrul acestui studiu.

Nu au fost aplicate optimizări manuale la nivel de sistem, precum overclocking, limitare explicită de frecvență sau control manual al afinității thread-urilor.

6. Modelare și implementare

Partea de modelare și implementare are rolul de a transpune problema teoretică a sortării într-un cadru experimental reproductibil.

6.1. Modelul general al experimentului

Pentru fiecare algoritm, experimentul urmează aceeași structură logică:

- 1) generarea sau citirea setului de date;
- 2) construirea structurii de date corespunzătoare;
- 3) pornirea cronometrării;
- 4) apelarea algoritmului de sortare;
- 5) oprirea cronometrării;
- 6) verificarea sau salvarea rezultatului;
- 7) înregistrarea timpului obținut.

Prin folosirea aceleiași scheme pentru toți algoritmi, rezultatele devin comparabile. Diferențele observate pot fi atribuite în principal algoritmului, structurii de date și distribuției inputului.

6.2. Structuri de date

Au fost folosite trei structuri de date:

- `array`, ca structură contiguă simplă;
- `vector`, ca structură contiguă din biblioteca standard C++;
- listă simplu înlănțuită, ca structură bazată pe pointeri.

Alegerea acestor structuri permite compararea dintre accesul direct și accesul secvențial, dintre stocarea contiguă și stocarea dispersată, precum și dintre costurile minime de indexare și costurile suplimentare de dereferențiere.

6.3. Algoritmi de complexitate quadratică

Bubble Sort, Selection Sort și Binary Insertion Sort sunt algoritmi utili didactic, deoarece permit observarea directă a modului în care un cost $O(n^2)$ devine rapid impracticabil. Deși Binary Insertion Sort reduce numărul de comparații prin căutare binară, costul mutării elementelor rămâne quadratic în cazurile generale.

Prin includerea acestor algoritmi, lucrarea stabilește o bază de comparație pentru metode mai eficiente.

6.4. Algoritmi de tip $O(n \log n)$

Merge Sort, QuickSort și IntroSort sunt reprezentanți ai clasei algoritmilor eficienți pentru volume mari de date. Merge Sort este robust și predictibil, QuickSort este eficient în practică prin partiționare in-place, iar IntroSort combină strategii diferite pentru a evita degradarea în cazuri nefavorabile [4, 6].

În această lucrare, aceste metode sunt comparate atât între ele, cât și cu implementările mai simple de tip $O(n^2)$.

6.5. Implementări manuale și implementări standard

O distincție importantă a lucrării este diferența dintre ideea algoritmică și implementarea concretă. De exemplu, `std::sort` este tratat ca etalon practic, deoarece implementările standard sunt optimizate în detaliu și respectă cerințe stricte de complexitate [3].

Implementarea manuală a IntroSort este importantă pentru înțelegerea algoritmului, însă nu trebuie confundată cu nivelul de optimizare al bibliotecilor standard. Această distincție este esențială pentru interpretarea corectă a rezultatelor.

7. Rezultate experimentale

Această secțiune prezintă principalele rezultate experimentale obținute. Valorile numerice provin din rapoartele generate în cadrul experimentului propriu [?].

7.1. Algoritmi de complexitate $O(n^2)$

Rapoartele pentru Bubble Sort și Selection Sort arată o degradare rapidă a performanței odată cu creșterea lui n . Pentru Bubble Sort, la $n = 10^5$, timpii ajung deja în zona zecilor de secunde, iar estimările pentru seturi foarte mari devin impracticabile.

De exemplu, pentru $n = 10^6$, Bubble Sort pe **vector** depășește 310 secunde, iar pentru $n = 10^7$ depășește 8 ore. Pentru $n = 10^8$, estimarea urcă la zeci de zile. Același tipar este observat și pentru Selection Sort, care rămâne în aceeași clasă de complexitate și produce costuri extreme pentru volume mari de date.

Din punct de vedere experimental, aceste rezultate validează direct faptul că algoritmi de ordin $O(n^2)$ sunt nerecomandați pentru volume medii și mari de date. Ei pot fi acceptabili doar pentru eșantioane foarte mici sau în contexte didactice.

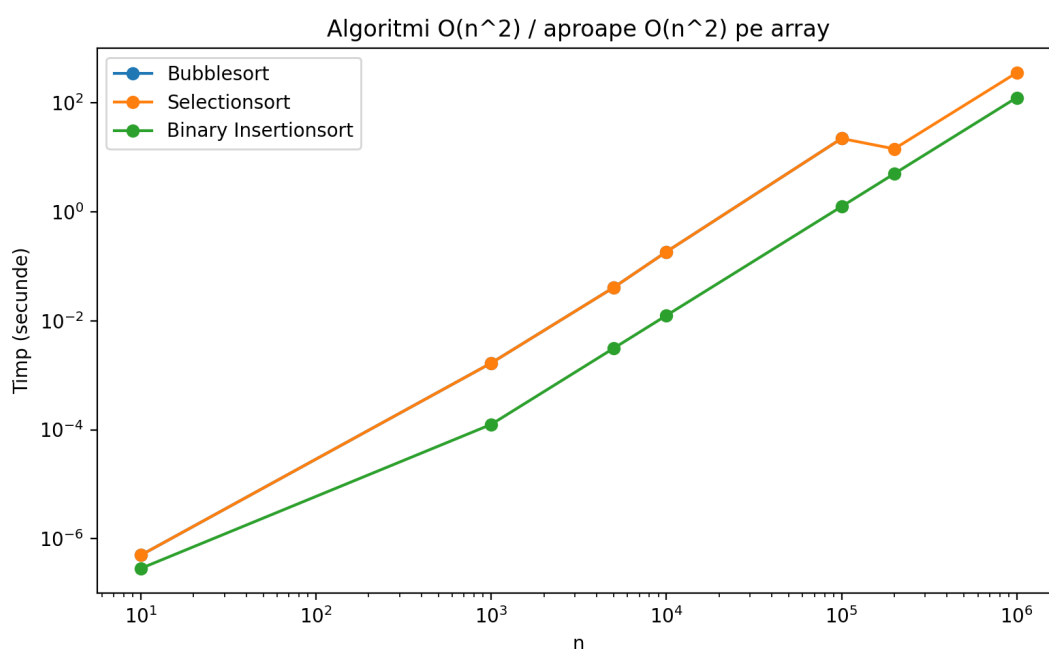


Figura 1: Evoluția timpului pentru Bubble Sort, Selection Sort și Binary Insertion Sort pe **array**, în scară logaritmică.

Graficul arată că Binary Insertion Sort aduce o ameliorare față de Bubble Sort și Selection Sort pe dimensiuni mici și medii, dar nu schimbă regimul fundamental al creșterii. Tendința rămâne incompatibilă cu date de ordin foarte mare.

7.2. Algoritmi de complexitate $O(n \log n)$

Merge Sort, QuickSort și IntroSort confirmă empiric superioritatea algoritmilor de ordin $O(n \log n)$. Pentru aceiași termeni de comparație, timpii rămân în zona fracțiunilor de secundă sau a câtorva secunde chiar și atunci când n ajunge la 10^7 sau 10^8 .

De exemplu, în contextul algoritmului QuickSort, pentru $n = 10^8$, timpul pe **vector** este aproximativ 4.10 secunde, iar pe **array** aproximativ 3.85 secunde. În raportul Merge Sort clasic, pentru același ordin de mărime, timpul pe **vector** este aproximativ 47.52 secunde, iar pe **array** aproximativ 17.29 secunde. În raportul IntroSort manual, pentru $n = 10^8$, timpul pe **vector** este de aproximativ 26.77 secunde, iar pe **array** de aproximativ 14.17 secunde.

Această diferență de ordin de mărime între $O(n^2)$ și $O(n \log n)$ este una dintre cele mai importante concluzii ale întregului experiment.

7.3. Comportamentul structurilor de date

Una dintre regularitățile cele mai clare din toate rapoartele este avantajul sistematic al **array**-ului față de **vector** și, mai ales, față de listă.

Motivele sunt:

- stocarea compactă în memorie;
- localitatea spațială bună;
- accesul direct la elemente;
- penalizare minimă pentru dereferențiere.

În majoritatea algoritmilor testați, **array**-ul obține cel mai bun timp total. Diferența față de **vector** este uneori moderată, dar constantă; diferența față de listă devine adesea foarte mare.

Lista simplu înlănțuită pierde din trei motive fundamentale:

- 1) accesul la elemente nu este direct, ci secvențial;
- 2) fiecare element este despărțit fizic în memorie de celelalte;
- 3) costurile de pointeri și dereferențiere se acumulează.

Prin urmare, chiar și atunci când algoritmul are teoretic aceeași complexitate asimptotică, implementarea pe listă poate fi mult mai lentă.

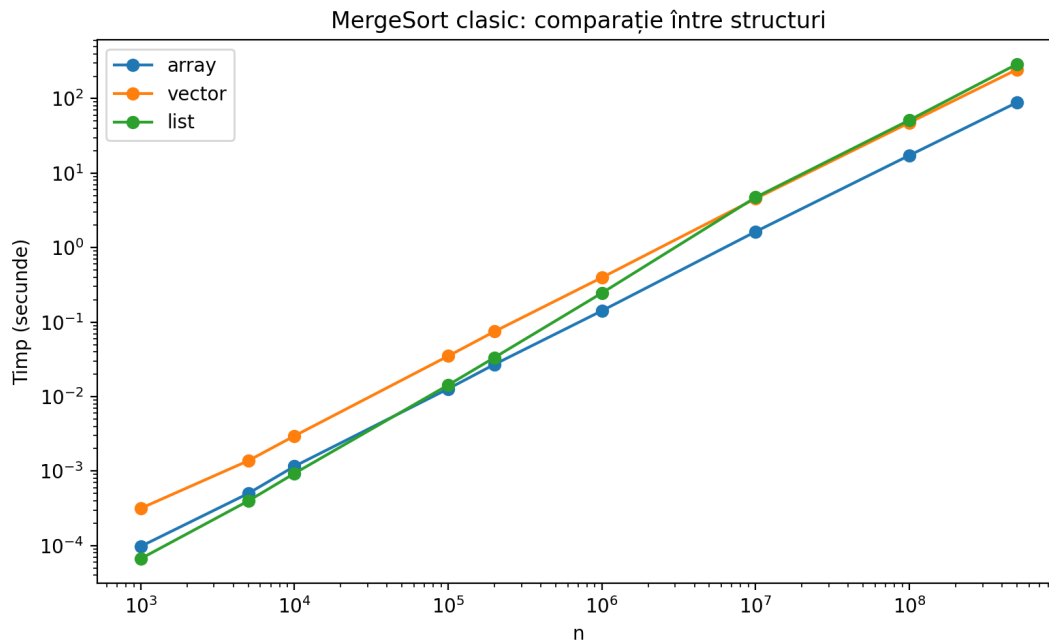


Figura 2: Merge Sort clasic: comparație între **array**, **vector** și listă simplu înlanțuită, în scară logaritmică.

Graficul evidențiază faptul că diferența dintre **array** și listă crește pe măsură ce n devine mare. Chiar dacă pe valori foarte mici lista poate părea competitivă, costurile reale de memorie și traversare o penalizează sever la scară mare.

8. Interpretarea rezultatelor

8.1. De ce Merge Sort pe listă pierde timp față de Merge Sort pe array

În teorie, Merge Sort funcționează bine și pe liste, deoarece interclasarea poate fi realizată elegant prin relink-area nodurilor. Totuși, rapoartele arată că implementarea pe listă este mai lentă decât cea pe **array**. Explicația principală nu ține de complexitatea asimptotică, ci de costul arhitectural al structurii.

Array-ul este cache-friendly: elementele consecutive sunt apropiate în memorie, iar procesorul preia eficient blocuri continue de date. În schimb, lista simplu înlanțuită presupune acces pointer-cu-pointer, cu dereferențieri repetate și slabă localitate de memorie. În plus, operațiile auxiliare precum împărțirea listei și traversările repetate introduc overhead suplimentar.

Experimental, diferența este clară. Pentru $n = 10^7$, Merge Sort pe **array** are aproximativ 1.64 secunde, iar pe listă aproximativ 4.73 secunde. Pentru $n = 10^8$, **array**-ul rămâne la aproximativ 17.29 secunde, în timp ce lista depășește 51.40 secunde. Pentru $n = 5 \cdot 10^8$,

array-ul este în jur de 88.51 secunde, iar lista urcă la aproximativ 289.41 secunde.

Rezultatul este că, deși Merge Sort este natural pentru liste din punct de vedere conceptual, în practică array-ul rămâne net superior pentru volume mari de date.

8.2. De ce Merge Sort paralel pe 16 thread-uri pierde față de QuickSort cu pivot random

Rezultatele experimentale arată că Merge Sort paralel pe 16 thread-uri nu depășește QuickSort-ul cu pivot random pentru volumele mari testate. De exemplu, pentru $n = 10^8$, Merge Sort paralel pe **vector** ajunge la aproximativ 14.01 secunde, în timp ce QuickSort pe **vector** rămâne în jur de 4.10 secunde. Pentru $n = 5 \cdot 10^8$, Merge Sort paralel pe **vector** este aproximativ 70.04 secunde, iar QuickSort aproximativ 23.50 secunde.

Din perspectivă de cercetare a performanței, această situație se explică prin următoarele:

- paralelizarea introduce costuri de creare și sincronizare a thread-urilor;
- interclasarea implică trafic suplimentar în memorie și copieri;
- QuickSort este in-place și necesită memorie auxiliară mult mai redusă;
- alegerea random a pivotului reduce probabilitatea de cazuri patologice și produce o partiționare eficientă în practică.

Așadar, deși paralelizarea reduce o parte din costul recursiei, ea nu elimină costul structural al algoritmului și nici overhead-ul execuției concurente. În practică, pentru implementarea testată, QuickSort rămâne mai eficient.

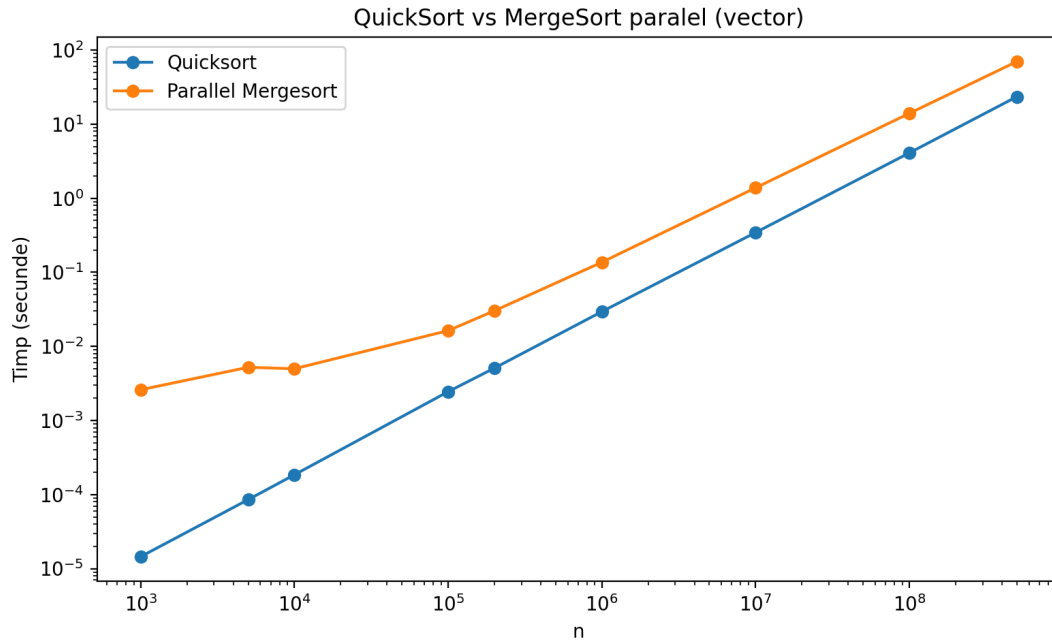


Figura 3: Comparație între QuickSort și Merge Sort paralel pe **vector**, în scară logaritmică.

8.3. De ce implementarea manuală de IntroSort este inferioară lui `std::sort`

Deși `std::sort` este asociat în practică cu strategii hibride de tip introspectiv, rezultatele arată că implementarea manuală este mult mai lentă. Pentru $n = 10^8$, implementarea manuală pe **vector** are aproximativ 26.77 secunde, iar pe **array** aproximativ 14.17 secunde. În schimb, implementarea optimizată din bibliotecile standard produce timpi de ordinul 4.10 secunde pe **vector** și 3.85 secunde pe **array** pentru aceeași scară experimentală.

Această diferență se explică prin faptul că `std::sort` nu este doar o idee algoritmică, ci o implementare industrială, rafinată la nivel de bibliotecă standard. Ea beneficiază de:

- praguri calibrate experimental pentru trecerea la insertion sort;
- partiționare foarte optimizată;
- management eficient al recursiei;
- compatibilitate bună cu optimizările compilatorului;
- reducerea overhead-ului pe ramuri și apeluri.

Prin contrast, implementarea manuală folosită în experiment este corectă conceptual, dar nu poate concura cu nivelul de rafinare al STL. Astfel, concluzia corectă nu este că

IntroSort este slab, ci că performanța unui algoritm depinde decisiv de calitatea concretă a implementării.

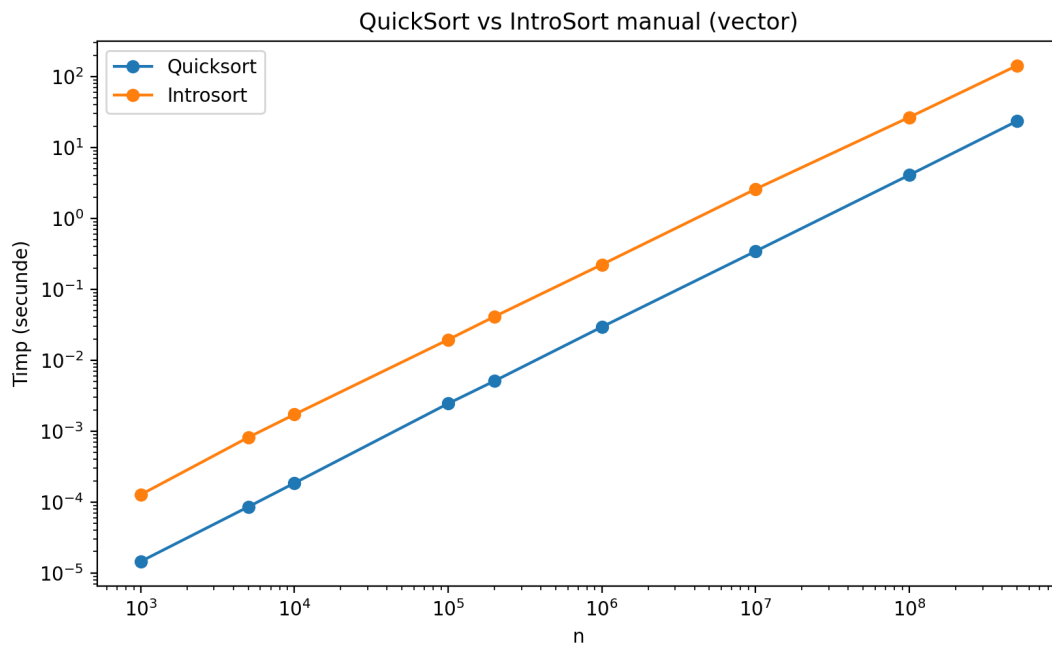


Figura 4: QuickSort vs IntroSort manual pe **vector**.

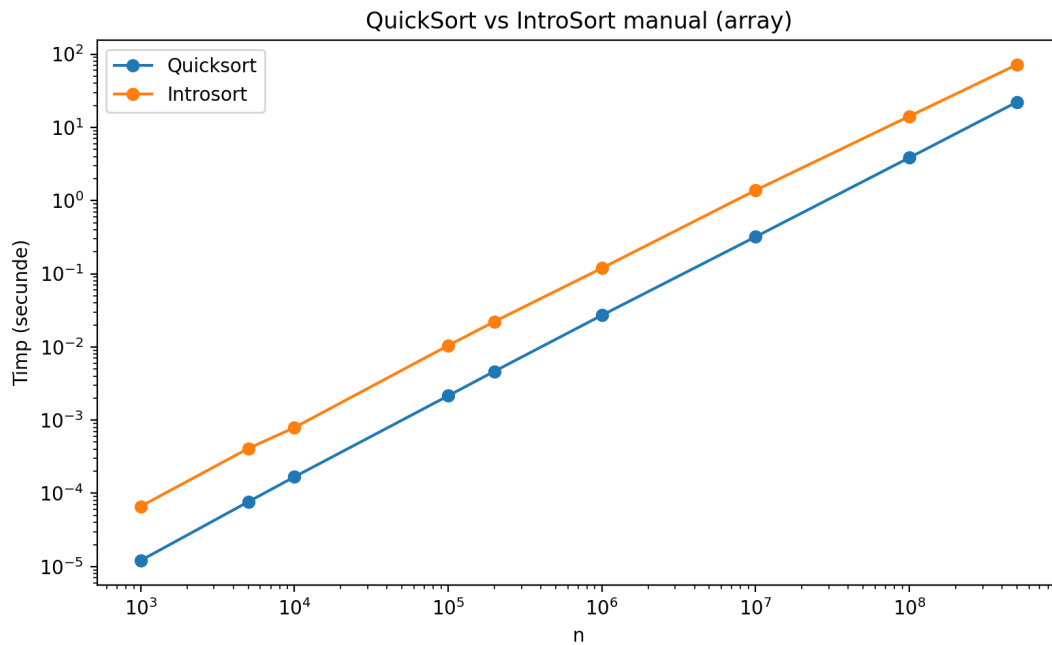


Figura 5: QuickSort vs IntroSort manual pe **array**.

8.4. Sensibilitatea la distribuția datelor

Seturile generate experimental au inclus nu doar distribuții random, ci și cazuri egale, aproape sortate, reverse, cu multe duplicate sau cu interval numeric restrâns. Această diversitate arată că aceeași dimensiune n poate produce timpi diferiți în funcție de forma inputului.

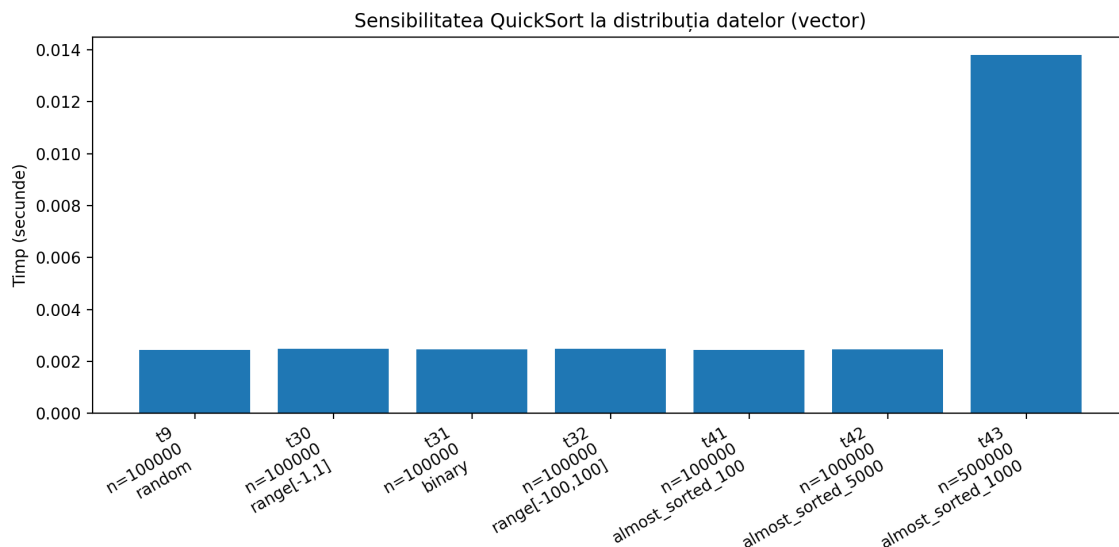


Figura 6: Sensibilitatea QuickSort la distribuția datelor pentru câteva teste reprezentative.

Forma distribuției influențează numărul de comparații, numărul de mutări și echilibrul partiționării. Din acest motiv, o analiză experimentală serioasă nu trebuie să folosească doar date random, ci și cazuri speciale, deoarece acestea pot evidenția limitele reale ale implementărilor.

8.5. Normalizarea tipurilor de date: conversia la întregi

O direcție interesantă de cercetare este transformarea diferitelor tipuri de date într-o reprezentare unificată de tip întreg.

Dacă pentru datele de tip `float` sau `double` este cunoscută precizia maximă de p zecimale, atunci fiecare valoare poate fi scalată prin înmulțire cu 10^p , iar apoi rotunjită și stocată ca întreg. În acest mod, sortarea se poate realiza doar pe valori întregi, evitând costurile și ambiguitățile asociate comparațiilor în virgulă mobilă.

Avantajele unei astfel de transformări sunt:

- uniformizarea codului de sortare;

- posibilitatea reutilizării aceluiași structuri și algoritmi pe un domeniu discret;
- control mai bun asupra ordonării și asupra reprezentării.

Limitarea majoră este riscul de overflow. Dacă valoarea scalată depășește intervalul tipului folosit, transformarea devine nesigură. În practică, `int` pe 32 de biți este adesea insuficient pentru intervale mari sau precizie ridicată, iar `long long` pe 64 de biți extinde mult domeniul, dar nu elimină complet riscul.

Pentru `char`, conversia la `int` este aproape naturală, deoarece caracterul are deja o reprezentare numerică internă. Din acest motiv, sortarea caracterelor poate fi interpretată ca o sortare de coduri numerice. Totuși, din punct de vedere al eficienței, câștigul real este limitat, deoarece comparațiile pe `char` sunt deja foarte ieftine. Beneficiul principal nu ar fi viteza brută, ci simplificarea și unificarea infrastructurii de testare.

Concluzia este că transformarea tipurilor în întregi poate fi utilă ca strategie de standardizare experimentală, dar trebuie controlată riguros pentru a evita overflow-ul și pierderea de informație.

9. Comparație cu literatura de specialitate

Rezultatele obținute sunt în acord cu literatura de specialitate în ceea ce privește diferența fundamentală dintre algoritmi de complexitate $O(n^2)$ și cei de complexitate $O(n \log n)$. Manualele clasice de algoritmică arată că metode precum Bubble Sort, Selection Sort și Insertion Sort au un cost quadratic în cazurile generale, ceea ce le limitează utilizarea practică la dimensiuni mici ale inputului [2, 5, 7].

Experimentele realizate confirmă această afirmație: pentru valori mari ale lui n , algoritmi $O(n^2)$ devin rapid impracticabili, în timp ce QuickSort, Merge Sort și IntroSort rămân utilizabili.

În cazul QuickSort, rezultatele sunt compatibile cu observațiile din lucrarea originală a lui Hoare [4], care a introdus un algoritm eficient de tip divide et impera. Implementarea cu pivot random reduce probabilitatea apariției cazurilor nefavorabile și produce rezultate foarte bune în practică.

În cazul Merge Sort, literatura evidențiază robustețea și predictibilitatea algoritmului, dar și costul suplimentar de memorie auxiliară [5, 2]. Rezultatele experimentale susțin această observație: Merge Sort are comportament stabil și previzibil, dar este penalizat de copieri și de traficul suplimentar în memorie.

În cazul IntroSort, rezultatele trebuie interpretate cu atenție. Musser [6] propune IntroSort ca algoritm hibrid care combină avantajele QuickSort cu garanțiile HeapSort.

Totuși, implementarea manuală testată în această lucrare este inferioară implementărilor standard optimizate. Această diferență nu contrazice literatura, ci confirmă faptul că performanța finală depinde decisiv de rafinarea implementării.

Lucrarea lui Bentley și McIlroy [1] este relevantă tocmai în acest punct: o funcție de sortare performantă nu este doar rezultatul alegerii unui algoritm bun, ci și al unei inginerii atente a cazurilor particulare, a pragurilor, a partiționării și a detaliilor de implementare.

Prin urmare, contribuția lucrării de față constă în confirmarea experimentală a rezultatelor teoretice cunoscute, dar și în evidențierea unor factori practici suplimentari: structura de date, localitatea în memorie, distribuția inputului și calitatea implementării.

10. Direcții viitoare de cercetare

Rezultatele experimentale obținute în această lucrare nu epuizează analiza posibilă, ci deschid mai multe direcții de cercetare viitoare, relevante atât din perspectivă teoretică, cât și practică.

10.1. Analiza sensibilității algoritmilor la distribuția datelor

O primă direcție importantă de cercetare este studiul sistematic al sensibilității algoritmilor de sortare la distribuția datelor de intrare. Seturile de test utilizate în această lucrare au fost generate astfel încât să acopere mai multe tipuri relevante de distribuții: random, sortat, invers sortat, egal, aproape sortat, cu multe duplicate și cu interval numeric restrâns.

În cercetări viitoare, această direcție poate fi extinsă prin gruparea formală a testelor pe clase de distribuții și prin construirea unor profile de sensibilitate pentru fiecare algoritm.

10.2. Studiul efectului localității în memorie

Rezultatele obținute indică în mod repetat că structura de date are un impact major asupra performanței. În majoritatea algoritmilor testați, `array`-ul este mai rapid decât `vector`-ul, iar lista simplu înlănțuită este frecvent cea mai lentă structură, mai ales pentru volume mari de date.

O direcție viitoare de cercetare este corelarea explicită dintre timpii de execuție și caracteristicile de acces la memorie, în special pentru:

- structuri contigue;
- structuri bazate pe pointeri;

- algoritmi care fac copieri auxiliare frecvente;
- algoritmi in-place versus algoritmi cu memorie auxiliară.

10.3. Praguri optime și algoritmi hibrizi

O altă direcție relevantă este determinarea experimentală a pragurilor optime pentru algoritmi hibrizi. În cazul IntroSort, performanța depinde nu doar de ideea de bază, ci și de momentul în care algoritmul trece de la QuickSort la HeapSort, respectiv de dimensiunea subproblemelor pentru care este introdus un algoritm auxiliar, precum Insertion Sort.

O continuare firească a cercetării ar fi calibrarea acestor praguri în funcție de:

- dimensiunea subsecvențelor;
- tipul de date;
- structura de stocare;
- distribuția datelor;
- arhitectura sistemului.

10.4. Strategii alternative de alegere a pivotului în QuickSort

În implementările QuickSort utilizate, alegerea pivotului a fost făcută random, ceea ce a oferit rezultate foarte bune în practică. Totuși, strategia de alegere a pivotului rămâne un factor critic în stabilitatea și robustețea algoritmului.

Prin urmare, o direcție de cercetare viitoare este compararea experimentală între:

- pivot random;
- primul element;
- ultimul element;
- median-of-three;
- alte strategii euristice de selecție.

O asemenea comparație ar permite determinarea relației dintre alegerea pivotului, distribuția datelor și stabilitatea timpului de execuție.

10.5. Raportul dintre paralelizare și beneficiul real de performanță

O direcție de cercetare deosebit de importantă este investigarea raportului real dintre paralelizare și câștigul de performanță. Teoretic, pentru volume mari de date, o implementare paralelizată a lui Merge Sort poate deveni foarte competitivă. Totuși, în experimentele prezente, acest lucru nu s-a confirmat, iar QuickSort a rămas superior în practică pe multe dintre testele mari.

Acest rezultat sugerează existența unui decalaj între potențialul teoretic al paralelizării și comportamentul practic al implementării testate. Tocmai această discrepanță justifică aprofundarea ulterioară a subiectului, prin analiza unor factori precum:

- costul real al creării și sincronizării thread-urilor;
- pragul de la care paralelizarea devine rentabilă;
- distribuția sarcinilor între thread-uri;
- costul memoriei auxiliare și al interclasării;
- numărul optim de thread-uri pentru o anumită arhitectură hardware.

Această direcție nu trebuie tratată ca o invalidare a ideii de Merge Sort paralel, ci ca un semnal clar că implementarea practică și mediul de execuție influențează decisiv rezultatul final.

10.6. Păstrarea datelor pentru validare și cercetare ulterioară

Păstrarea rapoartelor brute reprezintă un pas metodologic esențial. Datele experimentale obținute nu trebuie privite doar ca ieșiri temporare ale unui program, ci ca material de cercetare reutilizabil.

Aceste date permit:

- replicarea experimentelor;
- verificarea consistenței între rulări;
- construirea de grafice de scalare;
- compararea versiunilor optimizate cu cele neoptimizate;
- investigarea impactului distribuției inputului;
- calibrarea pragurilor de skip sau a limitelor experimentale.

În mod ideal, setul de rapoarte trebuie păstrat ca bază empirică pentru cercetare ulterioară asupra optimizării implementărilor, comparații între paradigme de memorie și analiză statistică a stabilității performanței.

11. Concluzii finale

Pe baza întregului set de rapoarte, se pot formula următoarele concluzii de cercetare:

- 1) Algoritmii $O(n^2)$ sunt improprii pentru volume mari de date, indiferent de structură.
- 2) Algoritmii $O(n \log n)$ reprezintă alegerea corectă pentru seturi mari.
- 3) Array-ul este, în mod constant, cea mai eficientă structură pentru sortare în experimentele realizate.
- 4) Lista simplu înlănțuită suferă din cauza localității slabe în memorie.
- 5) Paralelizarea nu garantează superioritate practică; overhead-ul poate anula beneficiul.
- 6) Standardizarea datelor și păstrarea rapoartelor sunt esențiale pentru cercetare ulterioară.
- 7) Implementările STL trebuie tratate ca etalon practic, nu doar ca referință teoretică.
- 8) Performanța reală depinde nu doar de algoritm, ci și de implementare, structură de date și distribuția inputului.

În concluzie, analiza experimentală confirmă că alegerea algoritmului optim nu depinde doar de formula teoretică a complexității, ci de interacțiunea dintre:

- algoritm;
- structură de date;
- reprezentarea tipului de date;
- implementare;
- arhitectura de memorie;
- costul practic al operațiilor auxiliare;
- distribuția datelor de intrare.

Lucrarea arată că teoria algoritmilor rămâne fundamentală, dar trebuie completată de analiză experimentală. Doar prin combinarea celor două perspective se poate înțelege comportamentul real al metodelor de sortare în practică.

Bibliografie

- [1] Jon L. Bentley and M. Douglas McIlroy. Engineering a sort function. *Software: Practice and Experience*, 23(11):1249–1265, 1993.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [3] cppreference.com. `std::sort`. <https://en.cppreference.com/w/cpp/algorithm/sort>.
- [4] C. A. R. Hoare. Quicksort. *The Computer Journal*, 5(1):10–16, 1962.
- [5] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2 edition, 1998.
- [6] David R. Musser. Introspective sorting and selection algorithms. *Software: Practice and Experience*, 27(8):983–993, 1997.
- [7] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4 edition, 2011.
- [8] J. W. J. Williams. Algorithm 232: Heapsort. *Communications of the ACM*, 7(6):347–348, 1964.